

WikiPulse — Complete Project Notes & Interview Prep

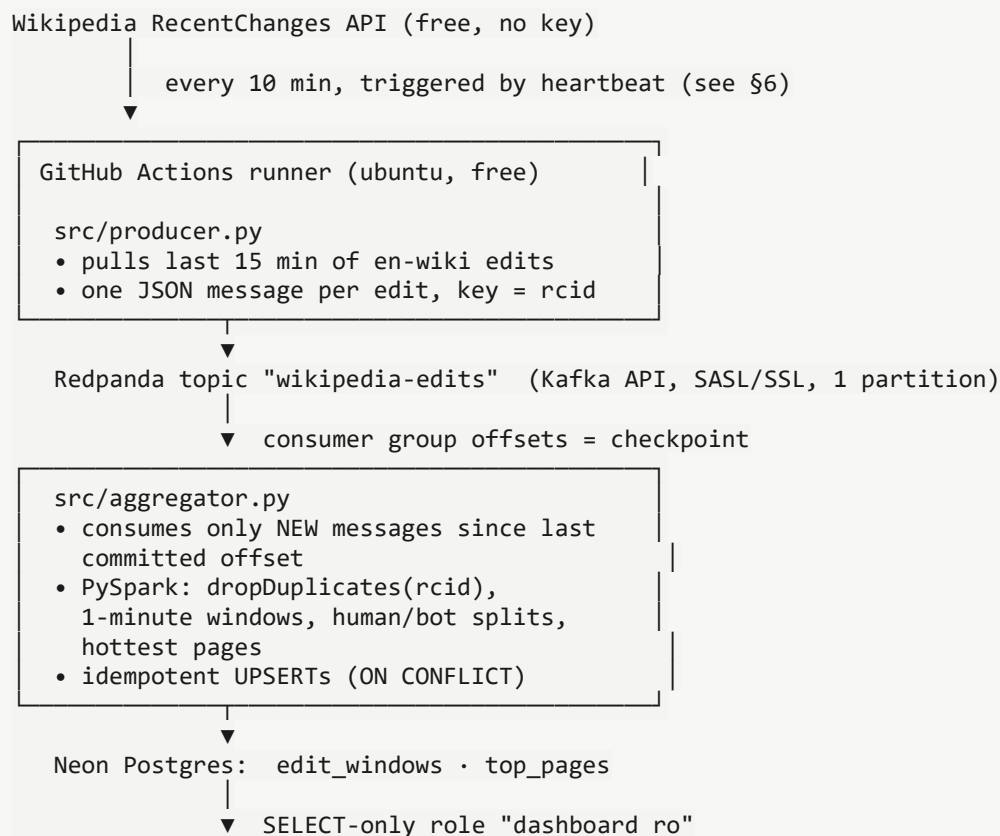
Live dashboard: <https://wikipulse-live.netlify.app> Source code: <https://github.com/vinn27/wikipulse> Built: Sep 27–30, 2026 · Cost: ₹0/month · Role: sole builder (design, code, ops)

1. What WikiPulse is (30 seconds)

Wikipedia is edited thousands of times every hour by humans and bots. WikiPulse is a **live analytics platform** that captures that activity automatically every 10 minutes, aggregates it with PySpark, stores it in Postgres, and serves it on a public, self-updating dashboard with Power BI-style interactions.

One-line pitch: *"Wikipedia's firehose flows through Kafka and PySpark into Postgres every 10 minutes on free cloud services — and that link is the live dashboard."*

2. Architecture & data flow



Next.js dashboard on Netlify

- /api/data → Neon, no-store
- client refreshes every 30 s
- also hosts the heartbeat function

Step-by-step data journey

1. **Heartbeat** (Netlify scheduled function) fires every 10 min → pings GitHub's `repository_dispatch` API with a least-privilege token → workflow `pipeline` starts.
2. **Producer** queries the MediaWiki RecentChanges API for the last 15 minutes (namespace 0 = article edits only), paginating 500 at a time, and publishes one JSON message per edit to Redpanda. `key=rcid` guarantees the same edit always lands on the same partition.
3. **Redpanda** (Kafka-compatible) retains messages — a durable, replayable buffer between ingestion and processing.
4. **Aggregator** (fixed consumer group `wikpulse-aggregator`) reads only messages after its last committed offset — Kafka does the checkpointing. PySpark dedupes by `rcid`, buckets edits into 1-minute windows, computes human/bot counts, unique editors, byte deltas, and hottest pages, then upserts into Neon. Tables are created idempotently (`CREATE TABLE IF NOT EXISTS`) so a fresh CI runner needs no setup.
5. **Dashboard** reads Neon through a SELECT-only role and renders; URL carries filter state; CSV export; 30-second auto-refresh.

3. Tech stack & why each piece

Layer	Tech	Why chosen
Orchestration	GitHub Actions (cron + <code>repository_dispatch</code>)	Public repo = free unlimited minutes; no server to run
Event bus	Redpanda (serverless)	Kafka-compatible, no credit card (Confluent is card-gated in India)
Processing	PySpark 3.5.9 (local[*])	Resume keyword + scales to a real cluster with the same code
Warehouse	Neon Postgres (free)	Serverless SQL, perfect for a star-ish serving layer
Dashboard	Next.js 16 + React + Tailwind, Recharts	Full control to hand-build BI interactions

Layer	Tech	Why chosen
Hosting	Netlify (free)	Vercel account was blocking new deployments; Netlify also hosts the heartbeat function
Language	Python 3.11 (CI) / 3.12 (local)	3.11 in CI = officially PySpark-supported combo

Deliberate design decisions

- **Micro-batch, not true streaming:** at ₹0 budget, a 10-minute cadence with exactly the same *architecture shape* (producer → broker → consumer with offsets) demonstrates all the streaming concepts at zero cost. Swapping to Spark Structured Streaming changes one file.
- **Overlap window (15 min pull every 10 min):** GitHub's clock jitters; the extra 5 minutes prevent gaps. Duplicates are absorbed by `rcid` dedup — correctness by idempotency, not by fragile timing.
- **Failing loudly:** scripts exit non-zero → red Actions run → red badge. Silent failure is the real enemy.
- **Read-only DB role for the dashboard:** even if the site's env leaked, the key cannot write or delete.

4. Data model

```

edit_windows (1 row per minute)
  window_start TIMESTAMPTZ PK  -- the minute bucket
  edits, bot_edits, human_edits INTEGER
  unique_editors INTEGER       -- distinct editors within that minute
  new_pages INTEGER            -- brand-new articles
  bytes_added / bytes_removed BIGINT
  updated_at TIMESTAMPTZ       -- last upsert time

top_pages (hottest human-edited pages per minute)
  (window_start, page_title) PK
  edits INTEGER                -- >= 2 human edits in that minute
  net_bytes BIGINT              -- added - removed

```

Upserts are `ON CONFLICT DO UPDATE` — re-running a window converges to the same values (last write wins per fully-covered window).

5. Dashboard feature list

- Auto-written **insight sentence** (volume, edits/min, human share, busiest bucket, hottest page)

- 4 KPI tiles with sparklines, deltas vs previous period, ⓘ hover definitions
- Stacked human/bot chart, **minute** → **hour** → **day drilldown**
- **Click-to-cross-filter** (click a column → whole dashboard filters; ✕ chip or Esc clears)
- **Focus mode** (⌘) fullscreen with a full data-table twin (no tooltip-only values)
- Time range slicers (1h → 7d + All data), Human/Bots/All audience toggle
- Hottest-pages table: search, sort, Wikipedia deep links, share-of-top bars
- **CSV export** on both visuals (Excel-safe)
- **Shareable URLs** (?range=24h&aud=human)
- 30-second auto-refresh that holds the previous render (no skeleton flash)
- **Data-freshness banner** — amber warning when data is > 15 min old
- `/guide` page: full self-explanatory walkthrough for non-technical visitors

6. Everything we improvised (the war stories)

These are the project's real value — every one was diagnosed with evidence, not guesses.

1. **Confluent demanded a credit card (India).** → Switched to Redpanda serverless (no card, Kafka-compatible). If its trial ever ends: swap = credentials only.
2. **No free always-on VM without a card** → redesigned from "VM running Spark 24/7" to **serverless micro-batch on GitHub Actions** — same architecture shape, zero infra.
3. **Aggregator consumed 0 messages** though the producer delivered 84. → Kafka consumer-group join over SASL/SSL takes 10+ seconds; the poll loop gave up after 5 quiet seconds. Fix: patience window. Lesson: *consumer group handshake timing*.
4. **PySpark workers crashed on Windows** ("Python worker exited unexpectedly"). → Isolation test proved even `count()` crashed → machine-wide, not our code → known bug: **Spark 3.5.8 + Python 3.12 on Windows**. Fix: pinned 3.5.9. (Also fixed `JAVA_HOME` + `winutils.exe` along the way.)
5. **The silent data lie:** `bot_edits = 0` across 125 edits — statistically impossible (bots do 20–50%). → MediaWiki API sends flags as empty-string-when-true; `bool("") == False`. Fix: `"bot" in rc`. Lesson: *domain-sense catches bugs that tests and logs never complain about*.
6. **Vercel served 404 for every NEW deployment** (old portfolio fine, three fresh projects dead, verified from two networks). → Moved hosting to **Netlify**; first deploy worked.
7. **GitHub's cron dropped most scheduled runs** — `*/10` never fired once; even off-peak minutes fired ~1 run per 5 hours. → Added **external heartbeat**: Netlify scheduled function pings `repository_dispatch` every 10 min — API-triggered runs are never dropped. GitHub cron stays as backup.
8. **Token 403** even after "Actions: read/write" → **repository_dispatch also needs** Contents: read/write**. Both set → 204 Accepted.
9. **Dashboard failed silently when data stalled** (blank "Last 1h") → added freshness banner + informative empty states + insight sentence. Monitoring UX is part of the pipeline.
10. **Junk sites from wrong-directory deploys** → cleaned up; lesson: always deploy with explicit `--cwd`.

7. Cost & security

Cost: ₹0/month. GitHub Actions (public repo, free), Redpanda serverless, Neon free, Netlify free.

Security practices - No secrets in code or git — `.env` (git-ignored) + GitHub Actions secrets + Netlify env vars - Dashboard DB access through **SELECT-only role** (verified: DELETE → permission denied) - Heartbeat token is fine-grained: one repo, two permissions (Actions/Contents RW), 1-year expiry - Passwords shown once are stored once; the Desktop secrets file was superseded by `.env`

Maintenance (calendar it) - ~Sep 2027: renew/rotate `wikpulse-heartbeat` token (GitHub → Developer settings) - Watch the Actions badge — red = something expired (Redpanda trial is the most likely candidate; swap is config-only)

8. Interview FAQ — with speakable answers

Q1. Why did you put Kafka/Redpanda between the API and processing? "At this scale it's architectural honesty more than necessity — but it buys real things: a durable, replayable buffer (I can reprocess history by resetting offsets), decoupling (producer and aggregator fail independently), and a checkpoint mechanism — consumer-group offsets remember what's been processed. My aggregator literally reads 'everything since my last bookmark.'"

Q2. What happens if the aggregator crashes mid-run? "Offsets commit only when the consumer closes cleanly, so a crash means the messages get re-read next run — at-least-once semantics. That's why the load layer is idempotent: dedupe by `rcid` plus `ON CONFLICT` upserts keyed on the window. Replays converge to the same result; duplicates are a design input, not an accident."

Q3. How do you prevent duplicate data? "Three layers: the producer keys messages by `rcid` (same edit → same partition), Spark drops duplicate `rcids` per batch, and Postgres upserts on natural keys. The overlap window — I pull 15 minutes every 10 — creates duplicates by design, and idempotency absorbs them."

Q4. Why micro-batch instead of true streaming? "Zero budget set the constraint, so I kept the streaming architecture shape — producer, broker, consumer groups, offset checkpointing — on a 10-minute cadence, all free. The swap path is one file: Structured Streaming with watermarks instead of batch reads. I know what changes: I'd add watermarks for late data and think about exactly-once sinks."

Q5. Why PySpark and not just pandas? "Pandas would honestly do for ~800 events per run. I chose Spark because the transformation code — windows, aggregations — is identical to what runs on a cluster; only the master changes. It also let me hit and solve real Spark problems, like the version-worker crash and dedup semantics."

Q6. What was your hardest bug? "A column showed 0 bot edits across 125 Wikipedia edits. Nothing crashed, nothing logged an error — the number was just wrong, because bots do 20–50% of Wikipedia's edits. The API marks bot edits with an empty-string flag, and `bool('')` is

False. I caught it because the value was *statistically impossible*, not because anything complained. That's my core belief about data engineering: validations should include 'does this number make sense in the domain', not just 'did the job succeed'."

Q7. How do you validate data quality? "Record counts per stage (produced vs consumed), row-level identity (rcid), statistical sanity (bot share, edits/min in a plausible band), and idempotent convergence — running twice changes nothing. In my work experience I also did source-to-target reconciliation: counts, nulls, duplicates, consistency."

Q8. Why Redpanda specifically? "Confluent requires a credit card in India, and self-hosting Kafka needs an always-on VM I didn't want to pay for. Redpanda serverless is Kafka-protocol compatible with no card — and because it's protocol-compatible, every line of code is standard Kafka; switching brokers later is a config change."

Q9. Why is your dashboard's DB role read-only? "Least privilege. The dashboard only ever reads, so it gets a role that can only SELECT — I verified a DELETE gets 'permission denied'. If the hosting env ever leaks, the blast radius is reading public Wikipedia aggregates."

Q10. Your pipeline runs on a schedule that failed silently. What did you do? "GitHub's scheduler drops most scheduled runs under load — my `*/10` never fired once. I shifted to off-peak minutes (still unreliable), then added an external heartbeat: a scheduled function on Netlify pings the `repository_dispatch` API every 10 minutes, and API-triggered runs are never dropped. I verified end-to-end: heartbeat → run → fresh Postgres rows. The general principle: don't trust one trigger — add a second, independent one, and make staleness visible (my dashboard shows an amber banner when data ages past 15 minutes)."

Q11. How would you scale this to all of Wikipedia, 100× volume? "More partitions on the topic and a consumer group with multiple instances (partitions are the parallelism unit), a real Spark cluster instead of `local[*]`, and a raw bronze layer — land events to object storage as Parquet before aggregating, so reprocessing never hits the API again. Postgres aggregates stay small; raw volume goes to a lake. Adding latency budget: move to Structured Streaming."

Q12. Why Postgres and not a 'real' warehouse? "The serving layer is tiny — pre-aggregated minute windows. Postgres is the right tool for a small, query-shaped serving layer, free, and serverless. If the consumer list grew I'd add a cache in front of the API, and move heavy history to a columnar store."

Q13. What's the dashboard doing that's special? "I hand-built the BI experience instead of using a BI tool: slicers, minute→hour→day drilldown, click-to-cross-filter, focus mode with a table twin, CSV export. The interactions are React state over one JSON endpoint — and colors are colorblind-validated. It taught me far more about dashboard UX than dragging widgets would."

Q14. What would you improve next? "Raw event retention to Parquet/Iceberg (reprocessable history), an alerting rule on staleness (email/Slack when the banner condition trips), a proper DQ suite (great-expectations-style checks with the bot-share sanity check as the first rule), unit tests on transforms, and true streaming on a paid tier."

Q15. Explain Kafka offsets like I'm new. "The topic is a log — append-only, numbered. My consumer group's 'committed offset' is a bookmark the group keeps: 'I've read till line 4,096.'"

Next run starts at 4,097 — old messages are never re-read unless I deliberately reset the bookmark, which is exactly how you replay history."

Q16. Why does 'Last 1h' sometimes show nothing on your own dashboard? "Because it means *the most recent hour from now*, and when the pipeline stalled (my scheduler saga), no data existed in that window. I turned that confusion into a feature: the dashboard now explains its own freshness — amber banner, informative empty states, and a guide page. A dashboard that hides its data gaps lies; mine narrates them."

Doc lives at `docs/PROJECT_NOTES.md` in the repo — keep it in sync if the project evolves.